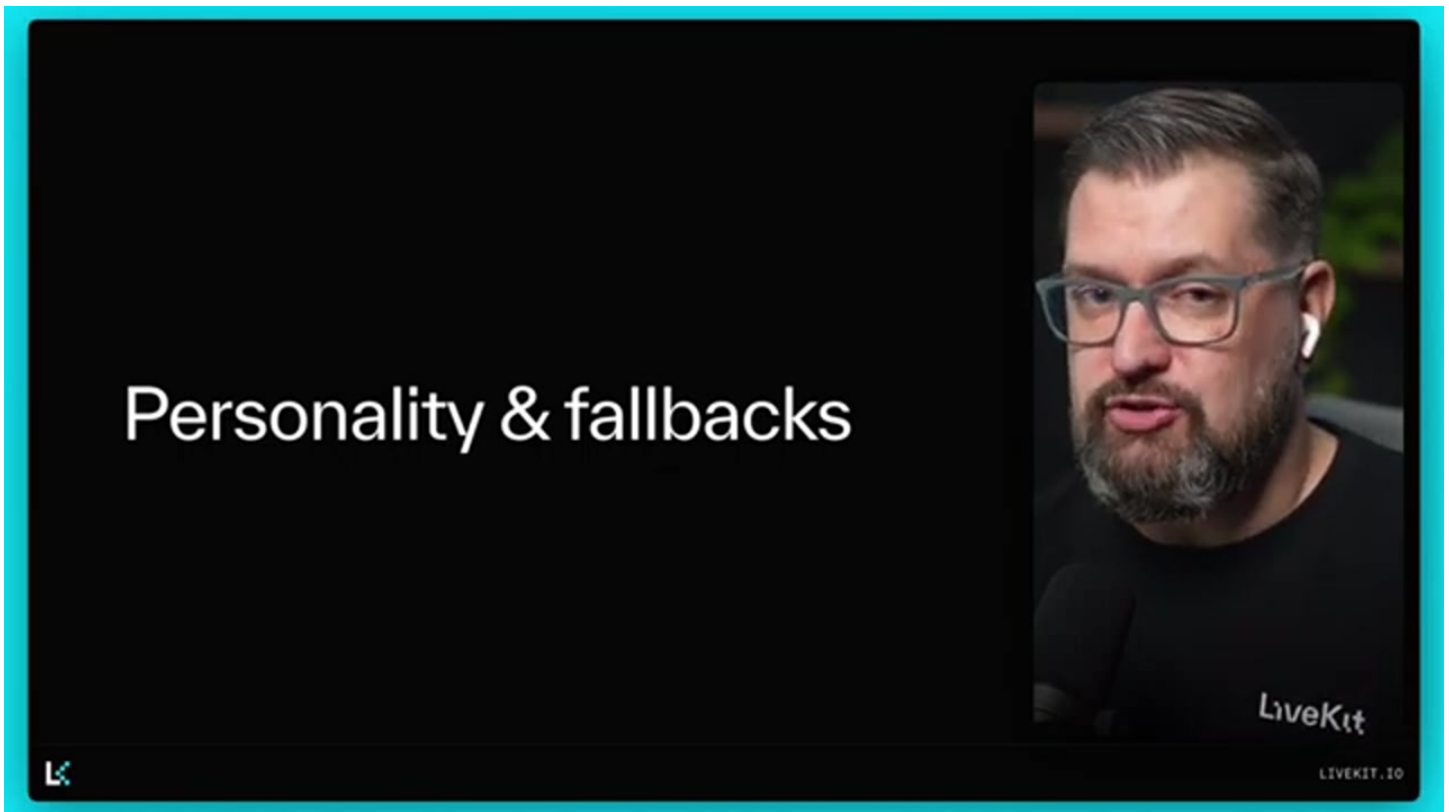


Lesson 3: Personality and Fallbacks

Section: LiveKit Voice AI Course

Summary: Coverage of Lesson 3: Personality and Fallbacks (18:46 - 26:11)

■ [18:46 - 19:06]



that it survives outages. Now that turntaking feels natural, it's time to make the agent sound on brand. In this lesson, you'll give the assistant a distinct personality, change its voice, and configure fallback adapters so that the agent survives provider outages. Voices aren't neutral speakers. How the

■ [19:06 - 19:26]

Voice + prompt = personality



LiveKit

LIVEKIT.IO

Voices aren't neutral speakers. How the LLM writes affects how the TTS sounds. So, first, match your phrasing to the voice. Shorter clauses and fewer parentheticals sound more natural when spoken. Second, be aware of dialect. Tailor spelling and idioms for authenticity, whether it's UK or US

Voice + prompt = personality

- Match phrasing to voice
- Dialect awareness



LIVEKIT.IO

authenticity, whether it's UK or US English. Third, use emotion control sparingly. Add subtle direction in system messages, but don't overdo it. And fourth, keep replies brief. Three sentences or fewer reduces latency and prevents your agent from monologuing. The system prompt and the voice work together as a team. They need to

Voice + prompt = personality

- Match phrasing to voice
- Dialect awareness
- Emotion control
- Keep replies brief



LIVEKIT.IO

together as a team. They need to complement each other. Now, your system prompt sets expectations for tone and boundaries. So, let's update the assistant class with a more specific personality. So, back in our uh agent.py in the assistant, let's update the instructions. So, I'm defining who this agent is. It's

■ [20:06 - 20:26]

```
tant(Agent):
    def __init__(self) -> None:
        self.__init__()
        self.instructions="You are an upbeat, slightly sarcastic voice AI for tech support. "
        self.instructions+="Help the caller fix issues without rambling, and keep replies under 3 sentences.",
```

```
entServer()
```

```
def _session():
    @contextmanager
    def entrypoint(ctx: JobContext):
        agent = AgentSession(
            model="assemblyai/universal-streaming:~",
            llm="openai/gpt-4.1-mini",
            tts="cartesia/sonic-3",
            vad=silero.VAD.load(),
            stt=n_detection=MultilingualModel(),
```

```
session.start()
```

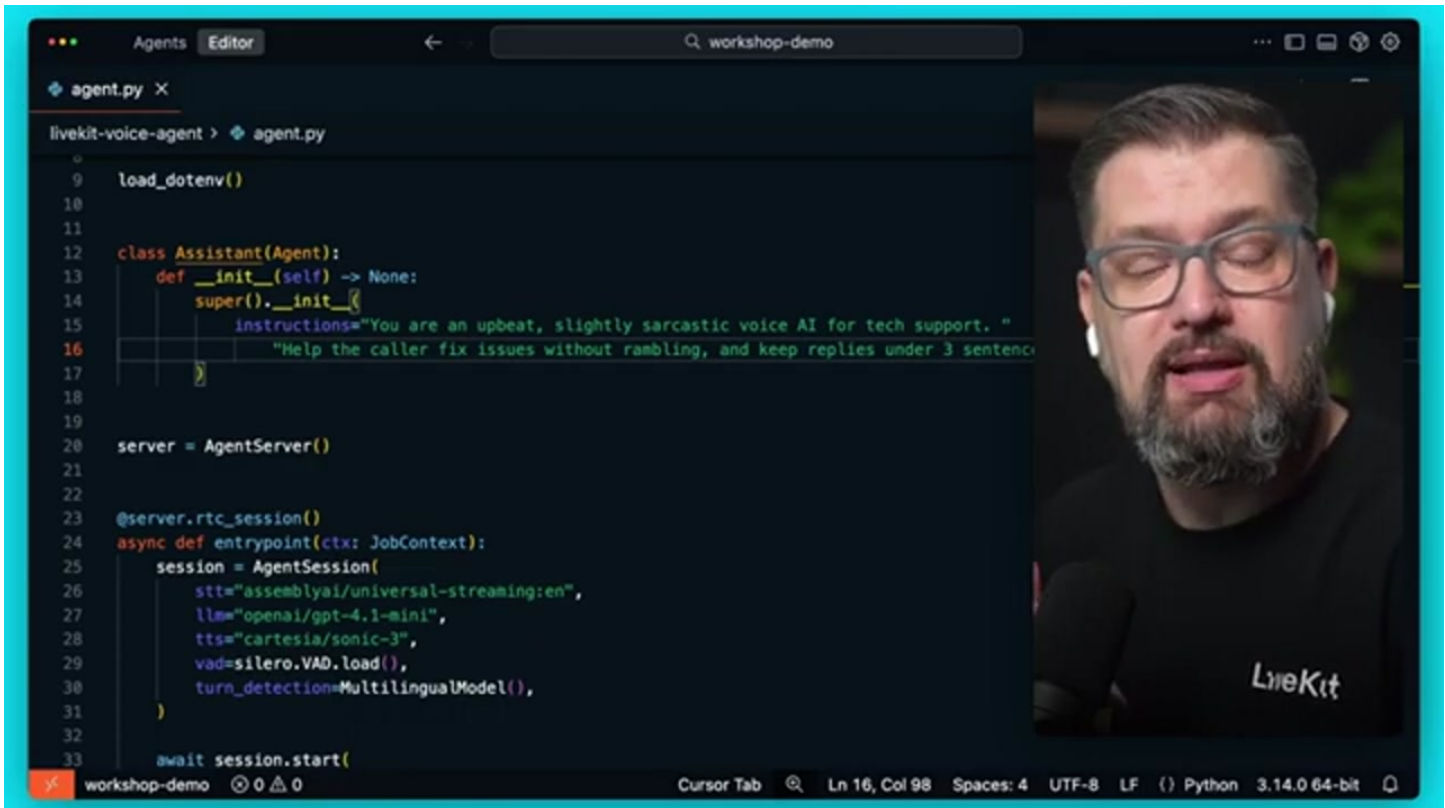
⊗ 0 △ 0

Cursor Tab 🔍 Ln 16, Col 98 Spaces: 4 UTF-8 LF {} Python 3.14.0 64



So, I'm defining who this agent is. It's an upbeat, slightly sarcastic voice AI or text support. It helps callers fix issues without rambling and keeps replies under three sentences. And you don't have to use this exact personality. Take some time to write instructions that make sense for your use case. Now, the key is being specific about tone, length, and boundaries.

■ [20:26 - 20:46]



The screenshot shows a video call interface. On the right, a man with a beard and glasses is speaking into a microphone. He is wearing a black t-shirt with the 'LiveKit' logo. On the left, a code editor window titled 'workshop-demo' displays the code for 'agent.py'. The code defines an 'Assistant' class that inherits from 'Agent', sets instructions, and configures an 'AgentSession' with specific STT, LLM, and TTS models.

```
9 load_dotenv()
10
11
12 class Assistant(Agent):
13     def __init__(self) -> None:
14         super().__init__(
15             instructions="You are an upbeat, slightly sarcastic voice AI for tech support. "
16             "Help the caller fix issues without rambling, and keep replies under 3 sentences."
17         )
18
19
20 server = AgentServer()
21
22
23 @server rtc_session()
24 async def entrypoint(ctx: JobContext):
25     session = AgentSession(
26         stt="assemblyai/universal-streaming:en",
27         llm="openai/gpt-4.1-mini",
28         tts="cartesia/sonic-3",
29         vad=silero.VAD.load(),
30         turn_detection=MultilingualModel(),
31     )
32
33     await session.start()
```

workshop-demo 0 0 0 Cursor Tab Ln 16, Col 98 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

about tone, length, and boundaries. Next, let's modify the TTS configuration to match the new personality. So, if we go down here, we can see in our agent session all of our uh definitions here. And what we're looking for is TTS text to speech. So, currently we're using Cartesia Sonic 3. And we can add a specific voice ID to this. If we

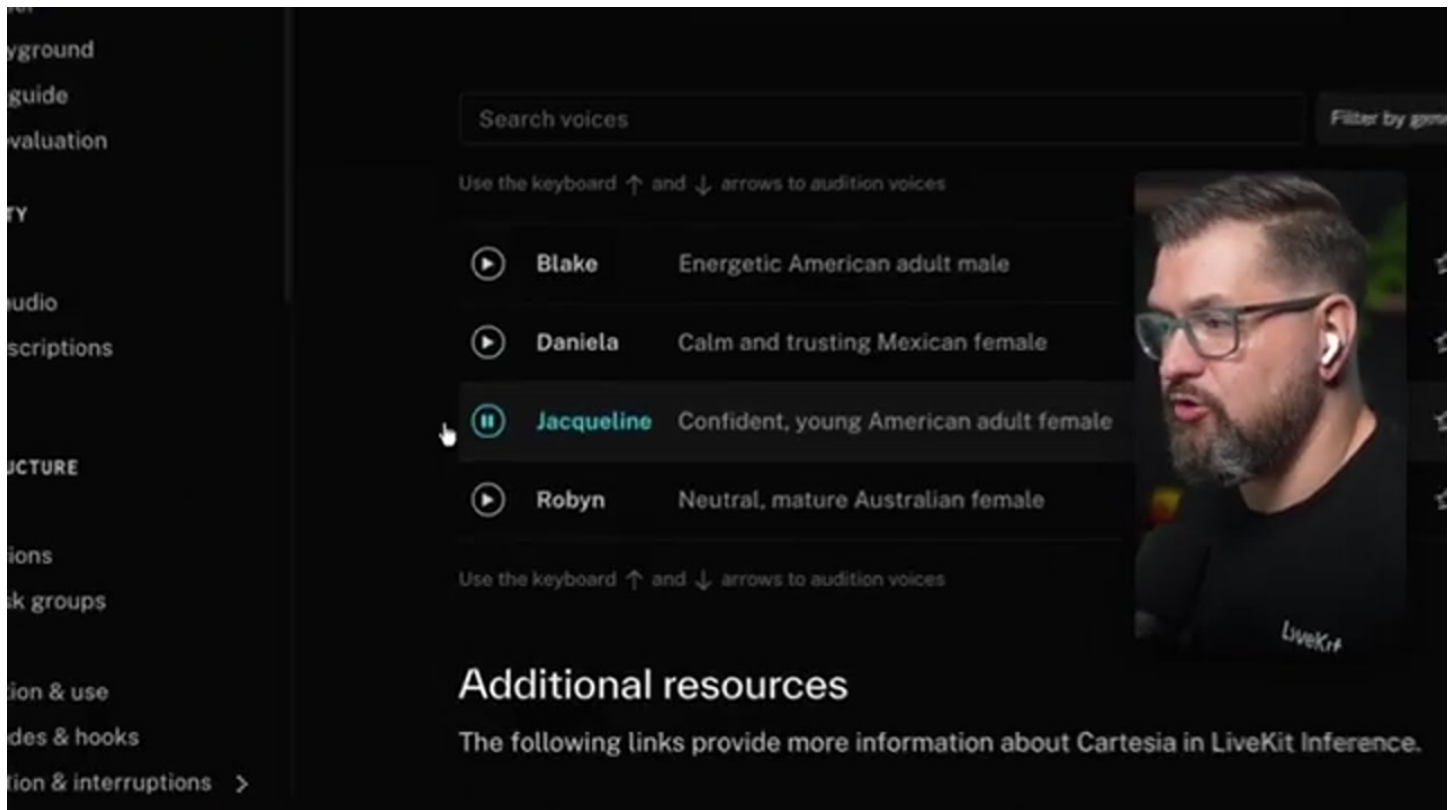
■ [20:46 - 21:06]

```
18
19
20 server = AgentServer()
21
22
23 @server.rtc_session()
24 async def entrypoint(ctx: JobContext):
25     session = AgentSession(
26         stt="assemblyai/universal-streaming:en",
27         llm="openai/gpt-4.1-mini",
28         tts="cartesia/sonic-3",
29         vad=silero.VAD.load(),
30         turn_detection=MultilingualModel(),
31     )
32
33     await session.start(
34         agent=Assistant(),
35         room=ctx.room,
36         room_options=room_io.RoomOptions(
```



add a specific voice ID to this. If we go over to the LiveKit docs and we can look at the specific Cartisia voices, we can actually play them here. So Blake is the default. >> Hi there. Nice to meet you. I'm looking forward to working with you. >> And there are others. >> Holaerte. >> Daniela is great for Spanish. Then there's Jacine.

■ [21:06 - 21:26]

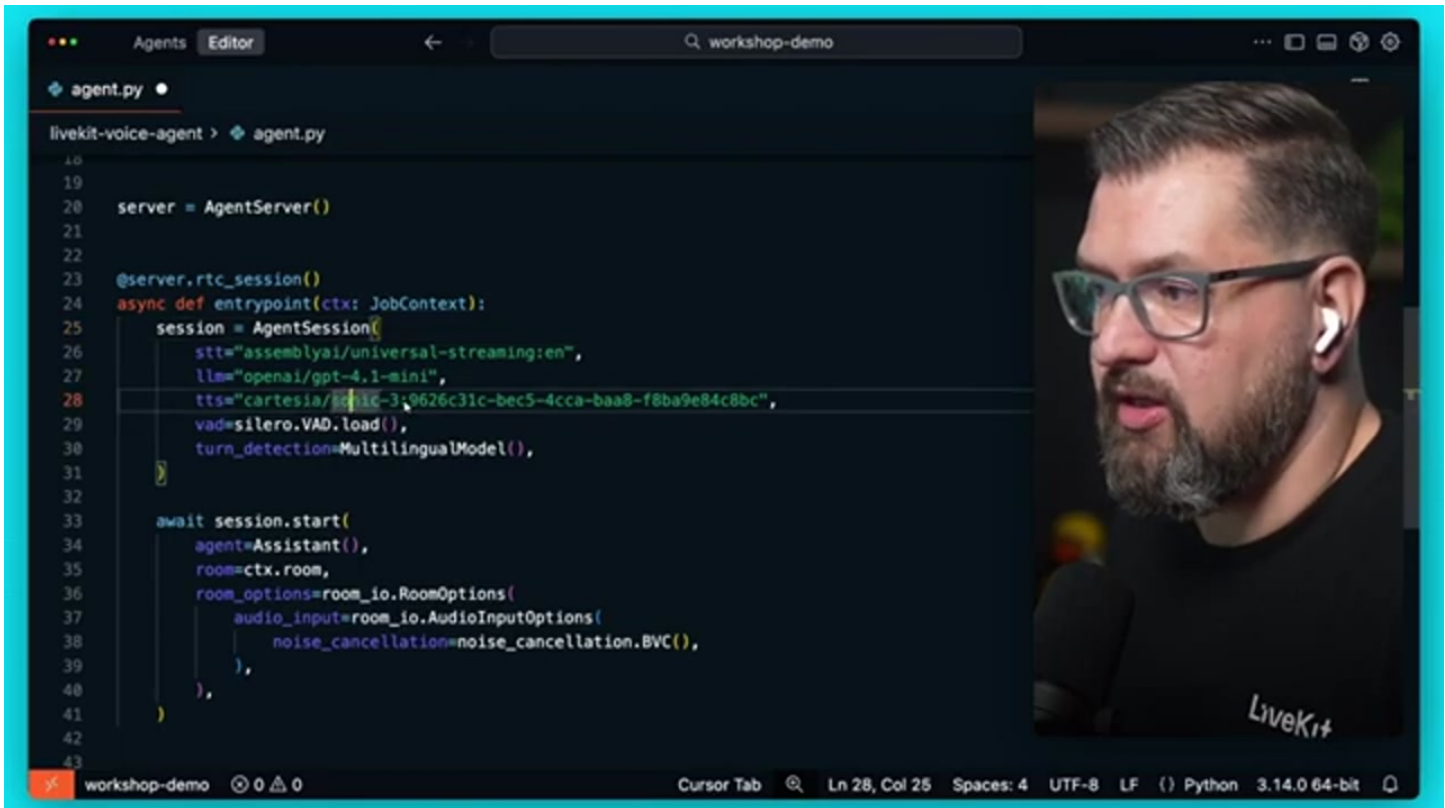


The screenshot shows a dark-themed web interface for LiveKit. On the left is a sidebar with navigation links: 'Background', 'Guide', 'Evaluation', 'RY', 'Audio', 'Descriptions', 'STRUCTURE', 'ions', 'k groups', 'ion & use', 'des & hooks', and 'ion & interruptions >'. The main content area has a 'Search voices' input field at the top right, with a 'Filter by genre' button next to it. Below the search bar is a list of voices, each with a play button icon, a name, and a description. The voices listed are: 'Blake' (Energetic American adult male), 'Daniela' (Calm and trusting Mexican female), 'Jacqueline' (Confident, young American adult female), and 'Robyn' (Neutral, mature Australian female). The 'Jacqueline' entry is highlighted with a green play button icon. Below the list is another instruction: 'Use the keyboard ↑ and ↓ arrows to audition voices'. To the right of the voice list is a video feed of a man with a beard and glasses, wearing a black shirt with the 'LiveKit' logo. Below the video feed, the text 'Additional resources' is displayed, followed by a paragraph: 'The following links provide more information about Cartesia in LiveKit Inference.'

Voice ID	Description
Blake	Energetic American adult male
Daniela	Calm and trusting Mexican female
Jacqueline	Confident, young American adult female
Robyn	Neutral, mature Australian female

there's Jacine. >> Hi there. Nice to meet you. I'm looking forward to working with you. >> Cool. So let's use Jacqueline. So any of these voices, if you like them, uh, just copy the voice ID. Let's go back and update our voice ID now. So right here, Cartisia Sonic 3. Let's just go ahead and update that voice ID there. So now it's Cartisia Sonic 3 and then colon and

■ [21:26 - 21:46]

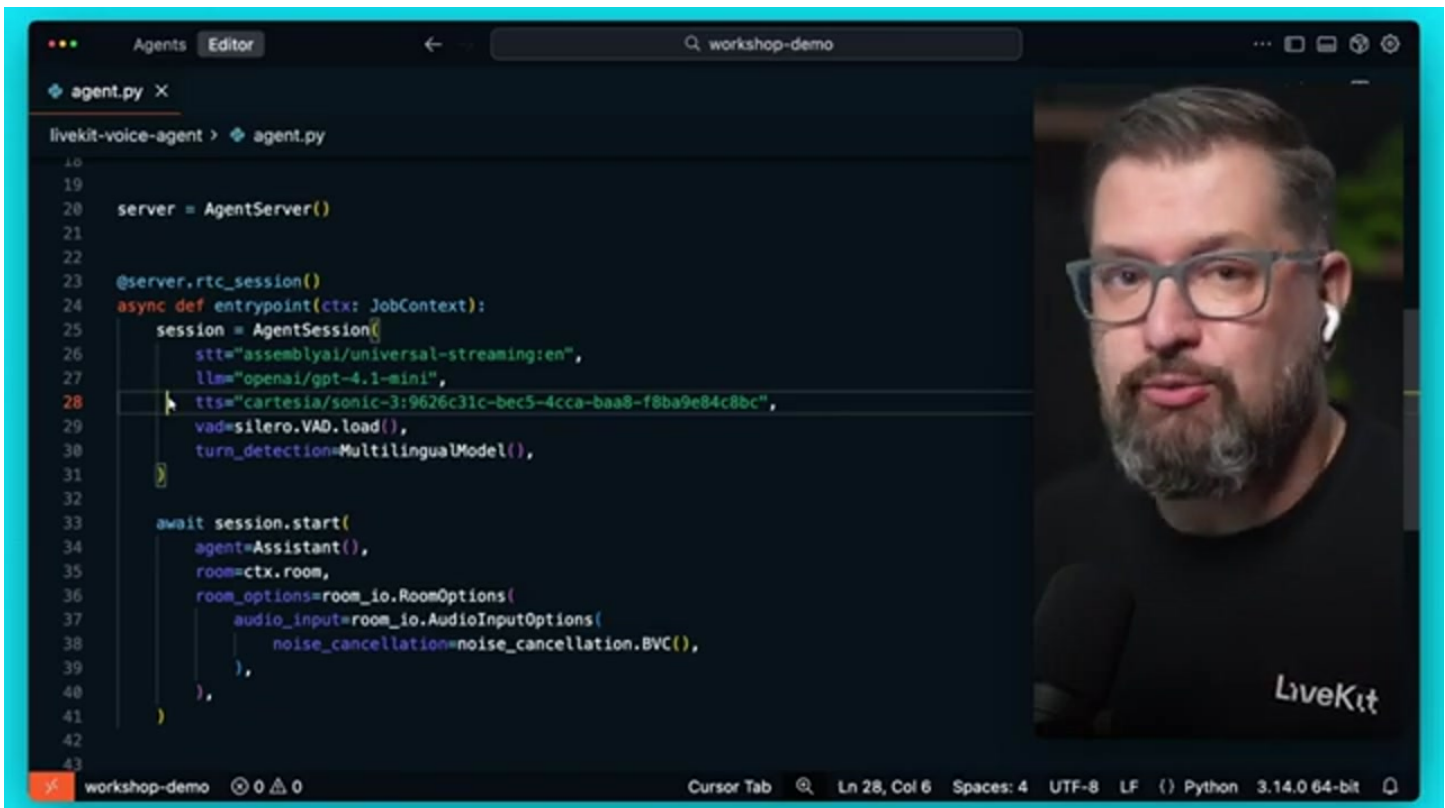


```
livekit-voice-agent > agent.py
10
19
20 server = AgentServer()
21
22
23 @server rtc_session()
24 async def entrypoint(ctx: JobContext):
25     session = AgentSession(
26         stt="assemblyai/universal-streaming:en",
27         llm="openai/gpt-4.1-mini",
28         tts="cartesia/sonic-3;9626c31c-bec5-4cca-baa8-f8ba9e84c8bc",
29         vad=silero.VAD.load(),
30         turn_detection=MultilingualModel(),
31     )
32
33     await session.start(
34         agent=Assistant(),
35         room=ctx.room,
36         room_options=room_io.RoomOptions(
37             audio_input=room_io.AudioInputOptions(
38                 noise_cancellation=noise_cancellation.BVC(),
39             ),
40         ),
41     )
42
43
```

workshop-demo 0 0 0 Cursor Tab Ln 28, Col 25 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

it's Cartesia Sonic 3 and then colon and then this is the actual voice ID here. And notice here that we have one model assigned to each component. But every provider has hiccups. When outages happen, you don't want your agent to stop working entirely. Production agents always need secondary providers. So livekits fallback adapters automatically

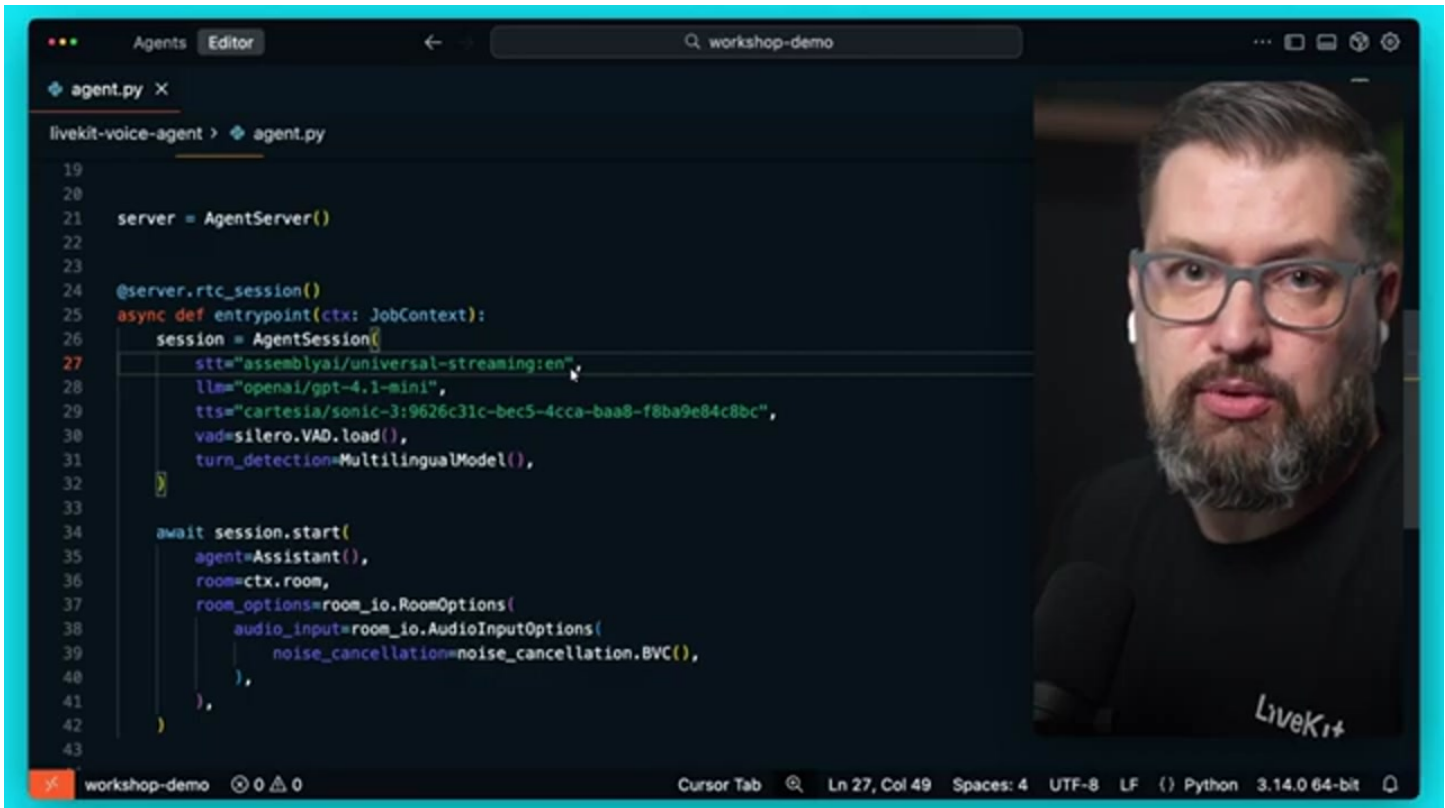
■ [21:46 - 22:06]

A screenshot of a video call interface. On the right side, there is a video feed of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. On the left side, there is a code editor window titled 'workshop-demo' showing a Python file named 'agent.py'. The code is for a LiveKit voice agent server. The editor has a dark theme and shows line numbers from 10 to 43. The status bar at the bottom indicates 'Cursor Tab', 'Ln 28, Col 6', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.14.0 64-bit'.

```
10
11
12
13
14
15
16
17
18
19
20 server = AgentServer()
21
22
23 @server rtc_session()
24 async def entrypoint(ctx: JobContext):
25     session = AgentSession(
26         stt="assemblyai/universal-streaming:en",
27         llm="openai/gpt-4.1-mini",
28         tts="cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc",
29         vad=silero.VAD.load(),
30         turn_detection=MultilingualModel(),
31     )
32
33     await session.start(
34         agent=Assistant(),
35         room=ctx.room,
36         room_options=room_io.RoomOptions(
37             audio_input=room_io.AudioInputOptions(
38                 noise_cancellation=noise_cancellation.BVC(),
39             ),
40         ),
41     )
42
43
```

lifekits fallback adapters automatically retry in priority order. So let's set up these fallback adapters. First let's go up to the top and we're going to import a few more things. We're going to import uh llm sttts and inference. And now let's go down. So instead of an individual component for each line, we're going to uh wrap each of these pipeline components with a fallback

■ [22:06 - 22:26]



```
19
20
21 server = AgentServer()
22
23
24 @server.rtc_session()
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
27         stt="assemblyai/universal-streaming:en",
28         llm="openai/gpt-4.1-mini",
29         tts="cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc",
30         vad=silero.VAD.load(),
31         turn_detection=MultilingualModel(),
32     )
33
34     await session.start(
35         agent=Assistant(),
36         room=ctx.room,
37         room_options=room_io.RoomOptions(
38             audio_input=room_io.AudioInputOptions(
39                 noise_cancellation=noise_cancellation.BVC(),
40             ),
41         ),
42     )
43
```

workshop-demo 0 0 0 Cursor Tab Ln 27, Col 49 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

pipeline components with a fallback adapter. So for st it's going to look something like this. We have st equals st.fallback fallback adapter and then we are going to add in our uh again we're using the lifekit inference so inference from model string and then we're passing in assembly AI as our primary and our

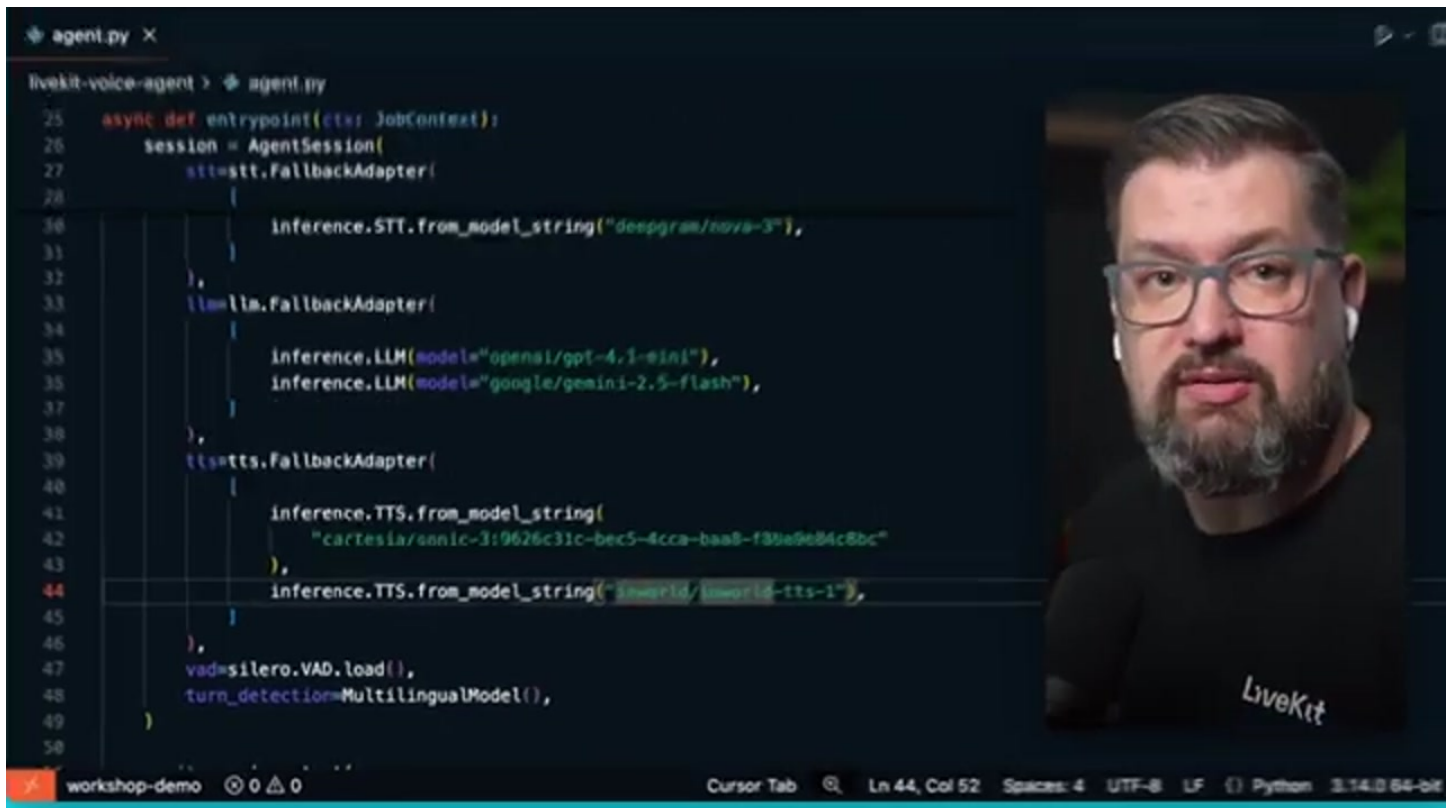
■ [22:26 - 22:46]

```
ctx: JobContext):  
    session(  
        backAdapter(  
            ence.STT.from_model_string("assemblyai/universal-streaming:en"),  
            ence.STT.from_model_string("deepgram/nova-3"),  
  
            ot-4.1-mini",  
            /sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc",  
            0.load(),  
            =MultilingualModel(),
```



in assembly AI as our primary and our secondary will be deepgram nova 3 so I'll go ahead and do the same for the llm and the tts all right so now we have each of these set up with fallbacks we have openai as the primary and then gemini as the as a fallback for our LLM we have cartisia as the primary TTS and in world as our secondary. So these

■ [22:46 - 23:06]

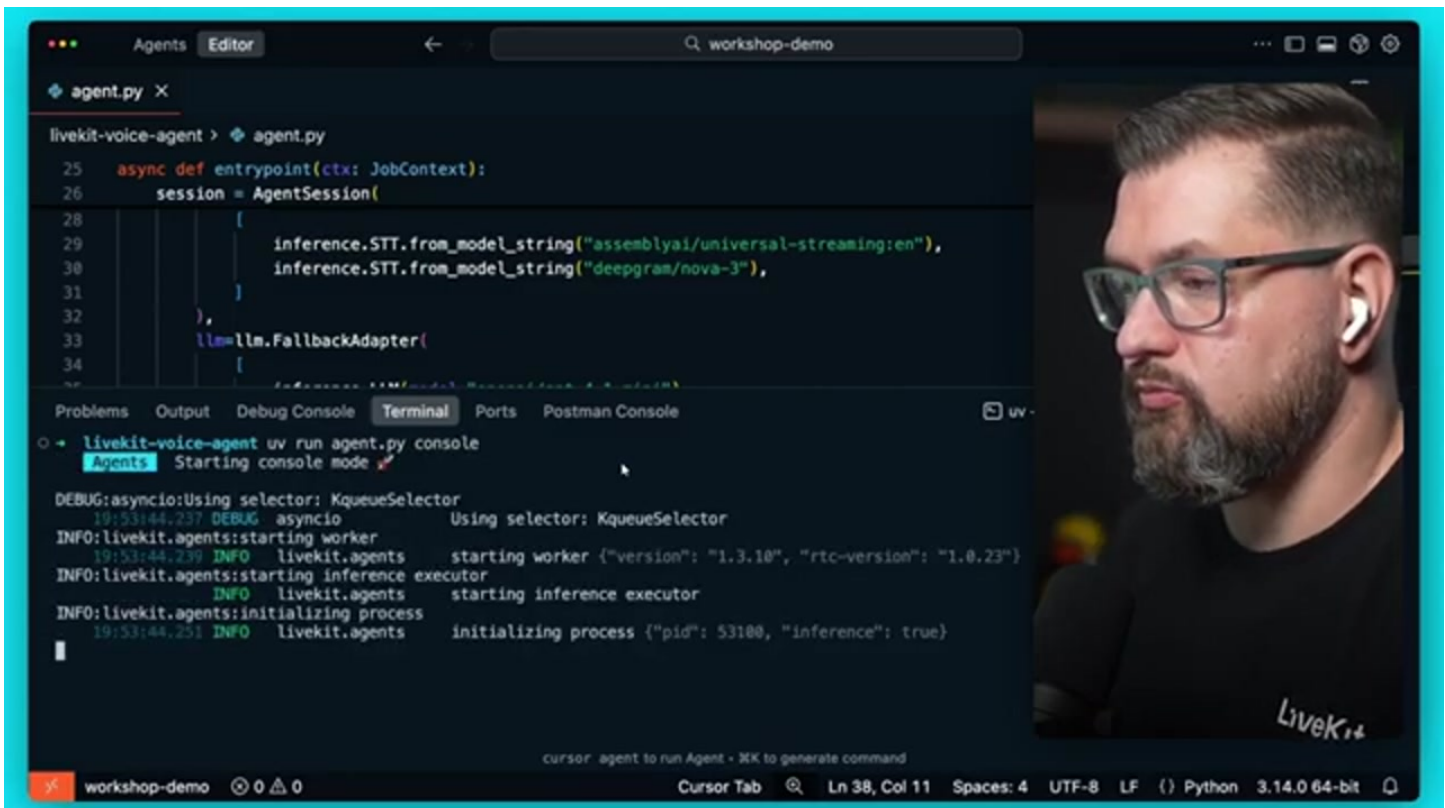


```
agent.py X
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
27         stt=stt.FallbackAdapter(
28             inference.STT.from_model_string("deepgram/nova-3"),
29         ),
30         llm=llm.FallbackAdapter(
31             inference.LLM(model="openai/gpt-4o-mini"),
32             inference.LLM(model="google/gemini-2.5-flash"),
33         ),
34         tts=tts.FallbackAdapter(
35             inference.TTS.from_model_string(
36                 "cartesia/annic-3:9626c31c-bec5-4cca-baa8-f38a9e84c8bc",
37             ),
38             inference.TTS.from_model_string("elevenlabs/eleven_tts_v2_5"),
39         ),
40         vad=silero.VAD.load(),
41         turn_detection=MultilingualModel(),
42     )
43
44
45
46
47
48
49
50
```

workshop-demo 0 0 0 Cursor Tab Ln 44, Col 52 Spaces: 4 UTF-8 LF Python 3.14.0 64-bit

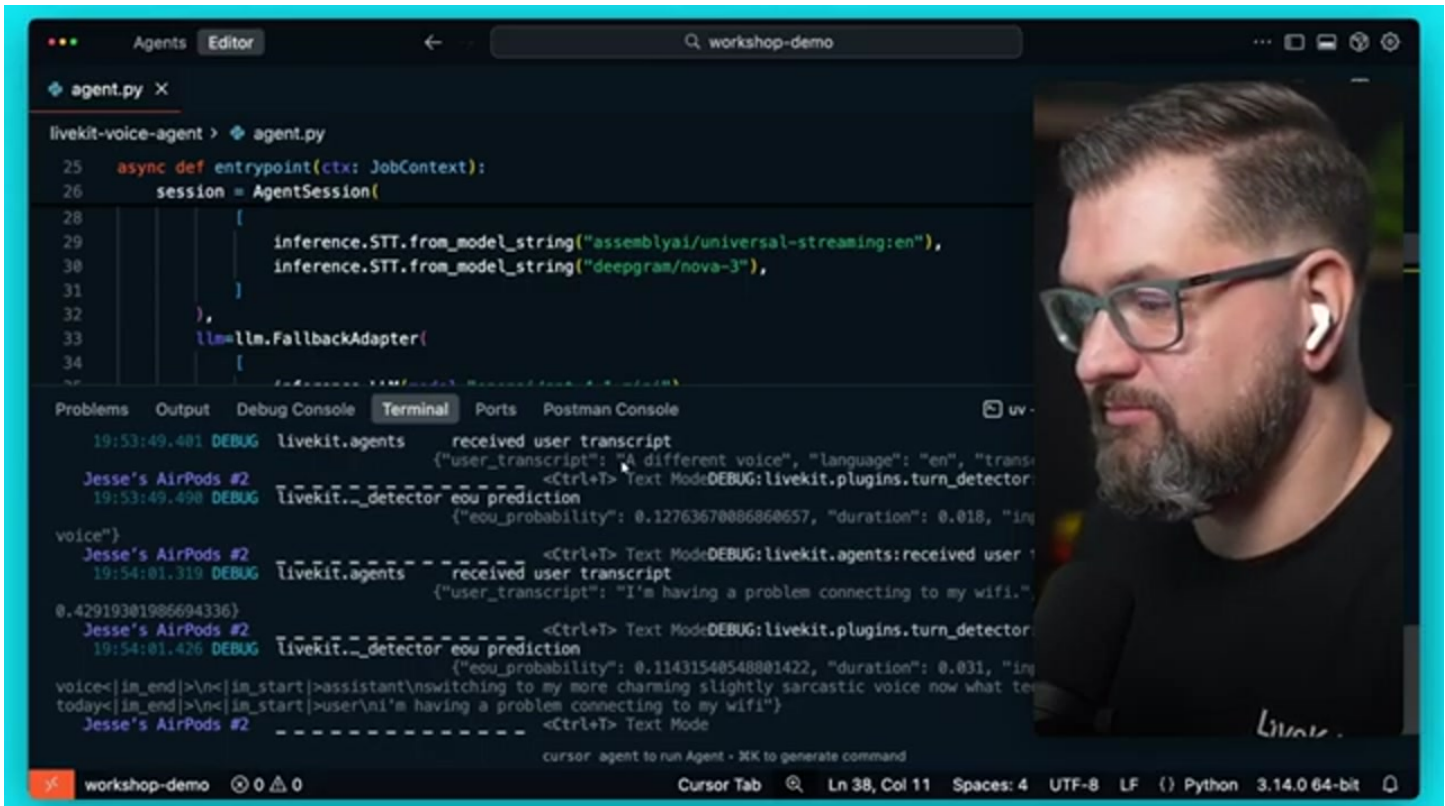
and in world as our secondary. So these adapters degrade gracefully. If the first provider fails, the session retries the next one without dropping the call. Your users never know anything went wrong. Let's test this out. Be sure that you saved the file. Open up our terminal and we'll do `uv run agents agent.py` and then console.

■ [23:06 - 23:26]



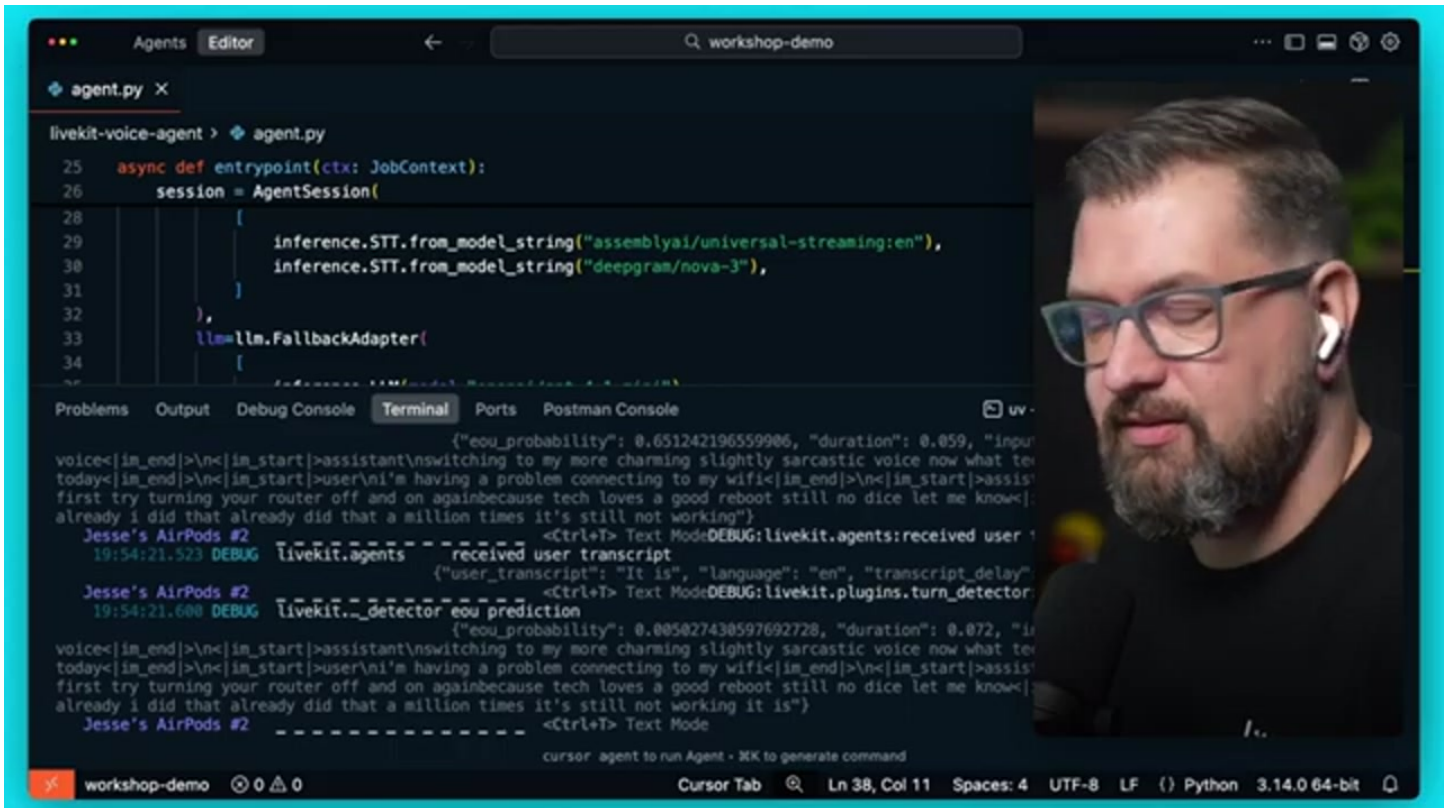
agent.py and then console. We should notice a different voice. Switching to my more charming, slightly sarcastic voice now. What tech trouble are we tackling today? >> Um, I'm having a problem connecting to my Wi-Fi. >> Uh, the classic Wi-Fi Houdini act. First, try turning your router off and on again because tech

■ [23:26 - 23:46]



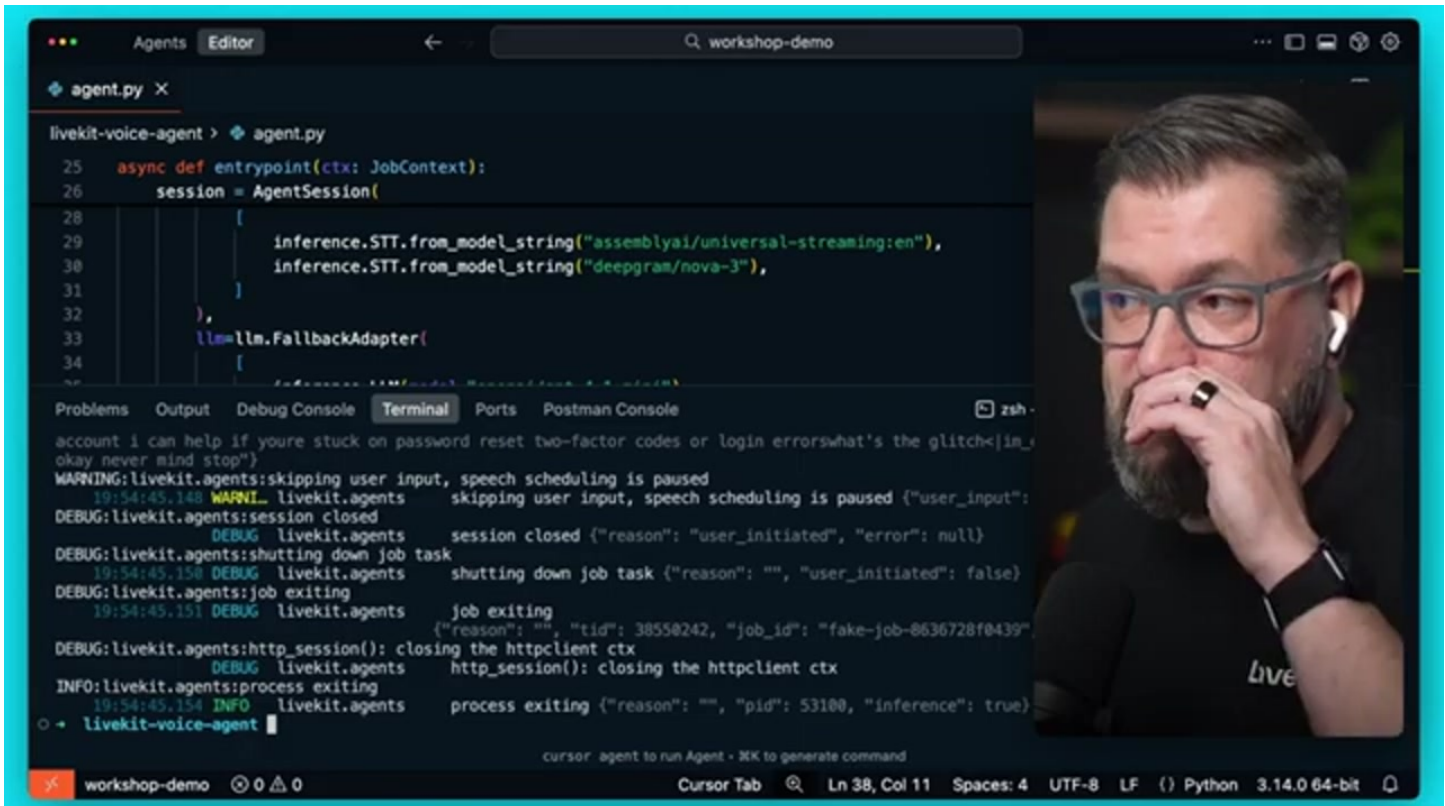
on again because tech >> I did that already. Did that a million times. It's still not working. >> Persistent router taps. Got it. Next. Check if your devices Wi-Fi is actually turned on. and try.

■ [23:46 - 24:06]



>> All right, your devices Wi-Fi is on. Good start. Try moving closer to the router to rule out >> something else. I'm actually um trying to log into my bank account. Can you help me with that? >> Sure thing. But for security reasons, I can't access your bank account. >> Ah, that's great. >> I can help if >> Okay, never mind. Stop. Okay, so seems to be working great. She's sarcastic.

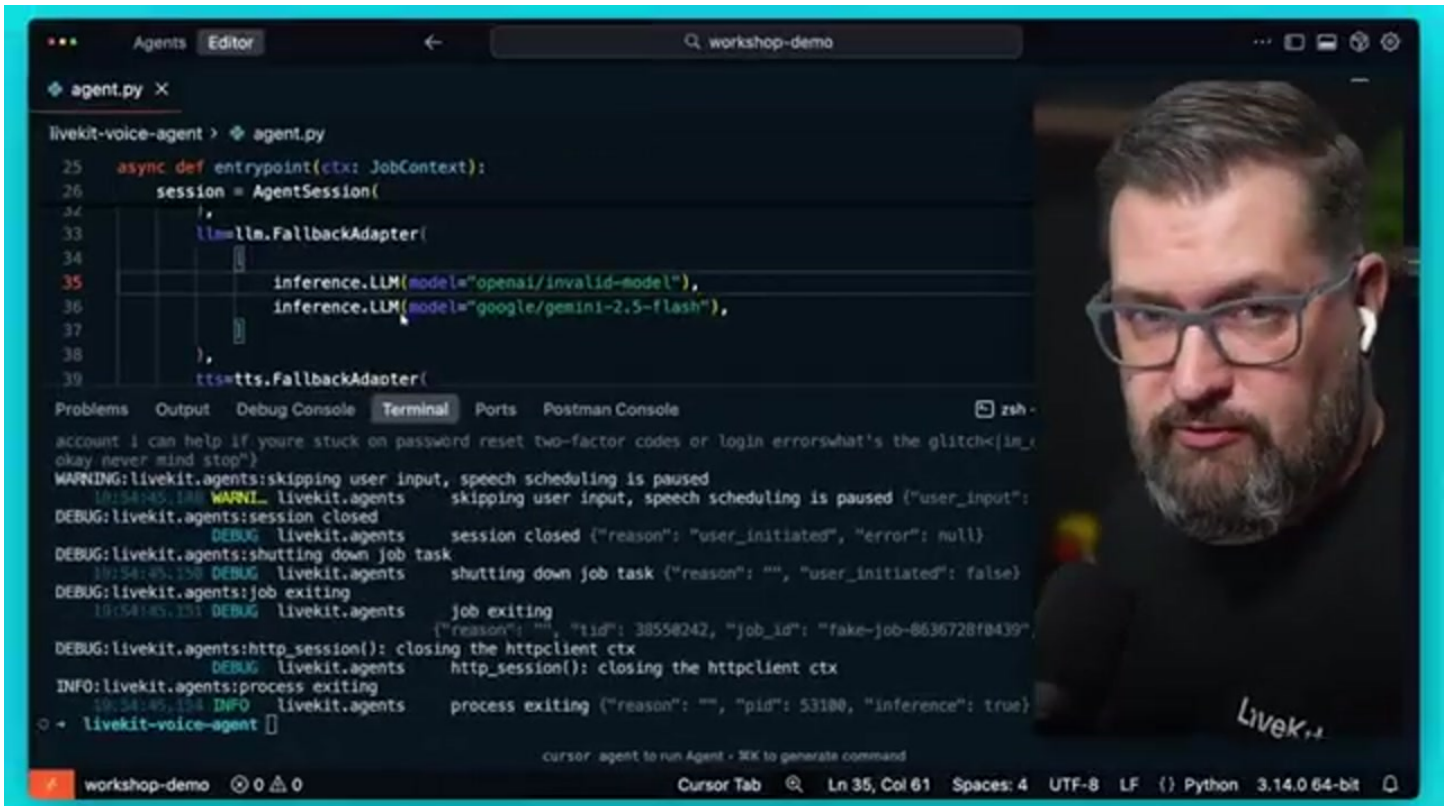
■ [24:06 - 24:26]



```
agent.py X
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
27         [
28             inference.STT.from_model_string("assemblyai/universal-streaming:en"),
29             inference.STT.from_model_string("deepgram/nova-3"),
30         ],
31     ),
32     llm=llm.FallbackAdapter(
33         [
34             inference.LLM.from_model_string("gpt-4o"),
35             inference.LLM.from_model_string("gemini-1.5-pro"),
36         ],
37     ),
38 )
39     await session.run(ctx)
40
Problems Output Debug Console Terminal Ports Postman Console
account i can help if youre stuck on password reset two-factor codes or login errorswhat's the glitch<jim_
okay never mind stop")
WARNING:livekit.agents:skipping user input, speech scheduling is paused
19:54:45.148 WARNING livekit.agents skipping user input, speech scheduling is paused {"user_input":
DEBUG:livekit.agents:session closed
DEBUG livekit.agents session closed {"reason": "user_initiated", "error": null}
DEBUG:livekit.agents:shutting down job task
19:54:45.150 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}
DEBUG:livekit.agents:job exiting
19:54:45.151 DEBUG livekit.agents job exiting
{"reason": "", "tid": 38550242, "job_id": "fake-job-0636728f0439"}
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
DEBUG livekit.agents http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
19:54:45.154 INFO livekit.agents process exiting {"reason": "", "pid": 53100, "inference": true}
○ + livekit-voice-agent
cursor: agent to run Agent - RK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 38, Col 11 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

to be working great. She's sarcastic. Um, we changed our voice. We've got our fallbacks. Let's test those fallbacks, actually. So, let's go up here and let's see. So, instead of GPT41, let's just do some uh in uh invalid model here. Uh, so we know that's not going to work. So, let's go ahead and test this out and see if it falls back to Gemini instead. So,

■ [24:26 - 24:46]



The screenshot shows a video call interface. On the right, a man with a beard and glasses is visible. On the left, a code editor displays the file `agent.py` with the following Python code:

```
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
33         llm=llm.FallbackAdapter(
34             inference.LLM(model="openai/invalid-model"),
35             inference.LLM(model="google/gemini-2.5-flash"),
36         ),
37     ),
38     tts=tts.FallbackAdapter(
39         inference.LLM(model="openai/invalid-model"),
40         inference.LLM(model="google/gemini-2.5-flash"),
41     ),
42 )
```

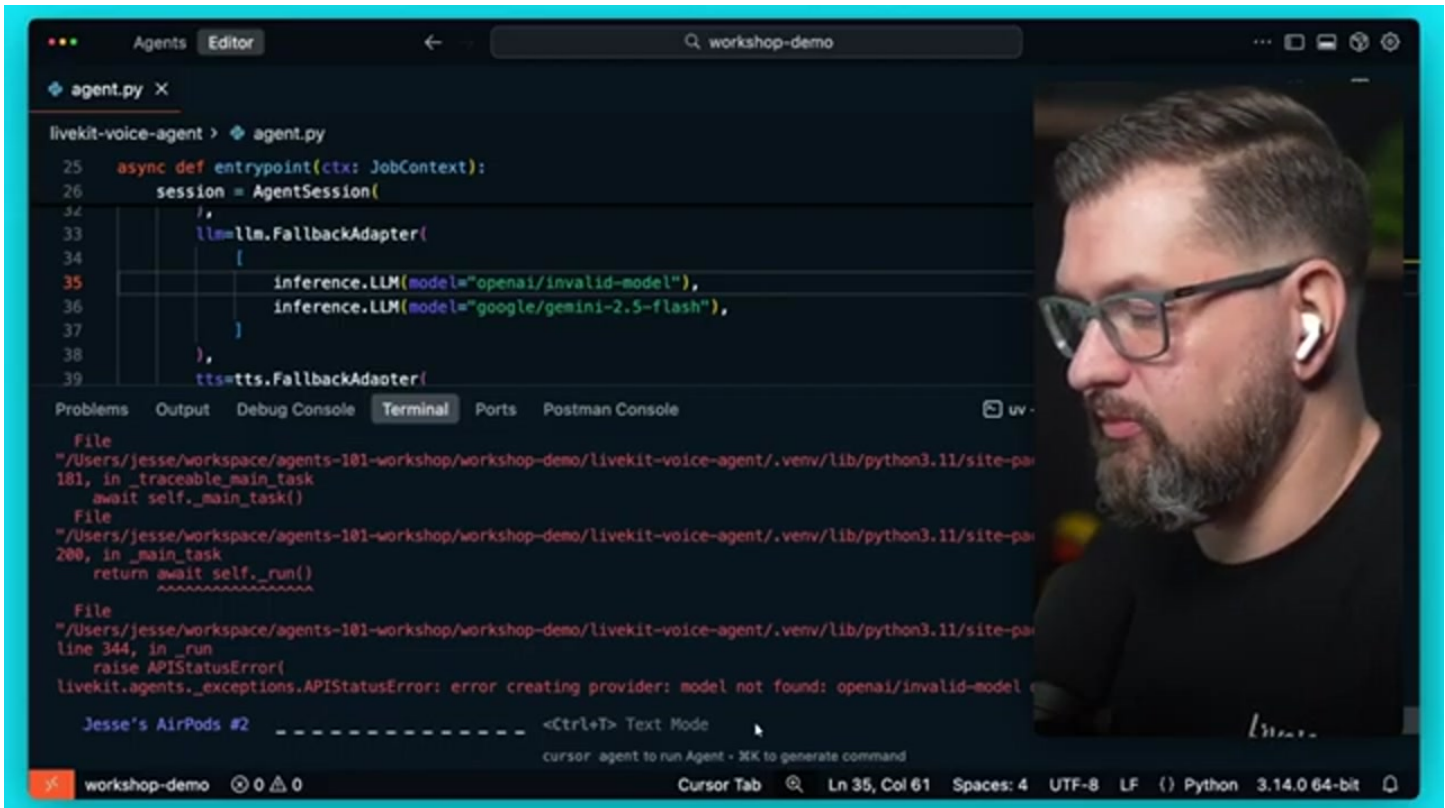
Below the code editor is a terminal window showing the following logs:

```
account i can help if you're stuck on password reset two-factor codes or login errorswhat's the glitch<[im_
okay never mind stop})
WARNING:livekit.agents:skipping user input, speech scheduling is paused
10:54:45.188 WARNING livekit.agents skipping user input, speech scheduling is paused ("user_input":
DEBUG:livekit.agents:session closed
DEBUG livekit.agents session closed ("reason": "user_initiated", "error": null)
DEBUG:livekit.agents:shutting down job task
10:54:45.190 DEBUG livekit.agents shutting down job task ("reason": "", "user_initiated": false)
DEBUG:livekit.agents:job exiting
10:54:45.231 DEBUG livekit.agents job exiting
{"reason": "", "tid": 38558242, "job_id": "fake-job-8636728f0439",
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
DEBUG livekit.agents http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
10:54:45.334 INFO livekit.agents process exiting ("reason": "", "pid": 53180, "inference": true)
+ livekit-voice-agent
```

The status bar at the bottom of the editor shows: `workshop-demo`, `Cursor Tab`, `Ln 35, Col 61`, `Spaces: 4`, `UTF-8`, `LF`, `() Python`, `3.14.0 64-bit`.

if it falls back to Gemini instead. So, we'll run a UV run agent. Hello. >> Hey there. Looks like you've stumbled into the land of helpful, slightly sarcastic tech support. What can I do for you? >> Um, yeah. Can you help me uh set up a brand new computer? >> Absolutely. Setting up a

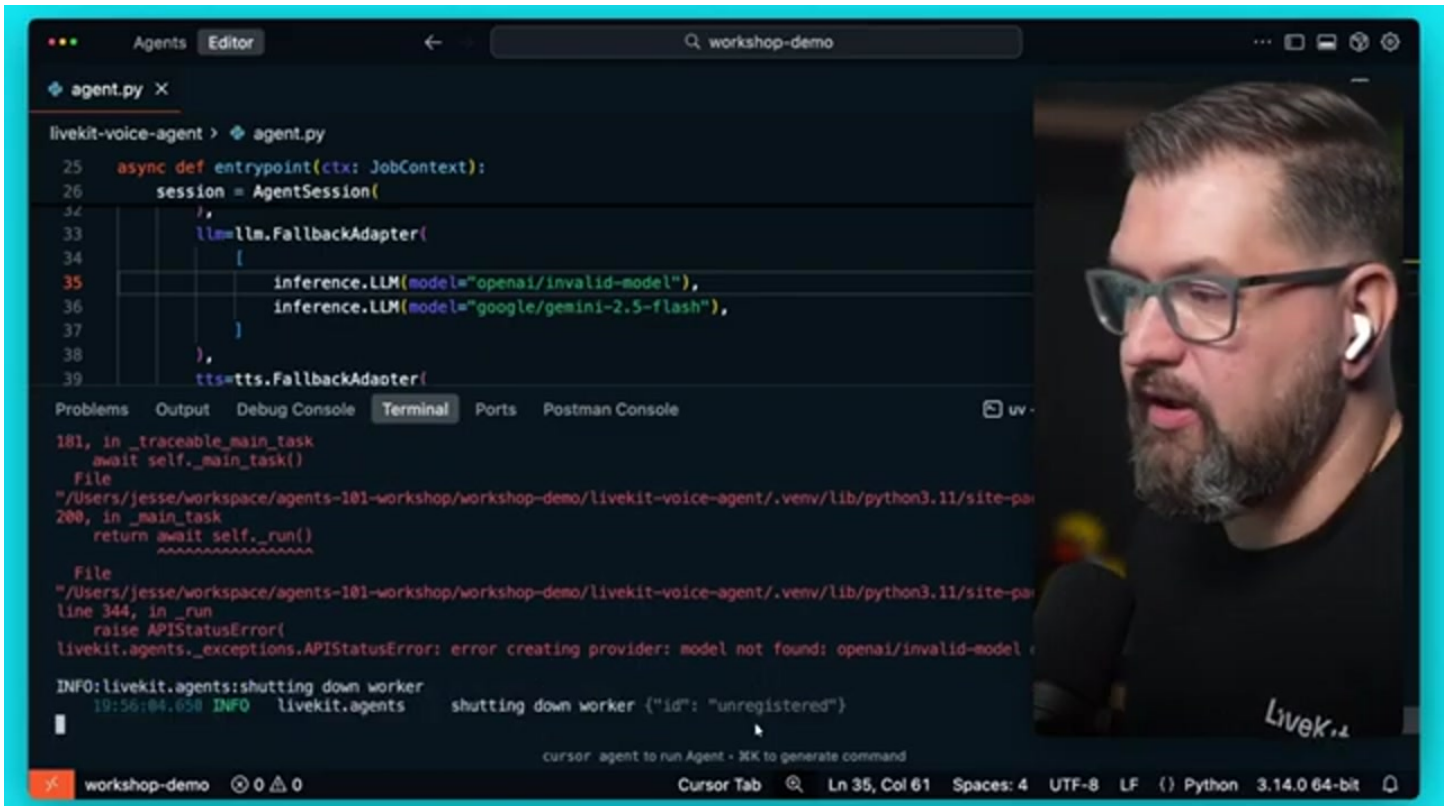
■ [24:46 - 25:06]



```
agent.py X
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
34         llm=llm.FallbackAdapter(
34             [
35                 inference.LLM(model="openai/invalid-model"),
36                 inference.LLM(model="google/gemini-2.5-flash"),
37             ]
38         ),
39         tts=tts.FallbackAdapter(
Problems Output Debug Console Terminal Ports Postman Console
File
"/Users/jesse/workspace/agents-101-workshop/workshop-demo/livekit-voice-agent/.venv/lib/python3.11/site-pa
181, in _traceable_main_task
    await self._main_task()
File
"/Users/jesse/workspace/agents-101-workshop/workshop-demo/livekit-voice-agent/.venv/lib/python3.11/site-pa
200, in _main_task
    return await self._run()
~~~~~
File
"/Users/jesse/workspace/agents-101-workshop/workshop-demo/livekit-voice-agent/.venv/lib/python3.11/site-pa
line 344, in _run
    raise APIStatusError(
livekit.agents._exceptions.APIStatusError: error creating provider: model not found: openai/invalid-model
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode
cursor: agent to run Agent - XK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 35, Col 61 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

>> Absolutely. Setting up a new computer is like unwrapping a present, >> but with more cables. To start, let's make sure you've got everything out of the box. >> Yeah. Yeah. Yeah. It's all out of the box, plugged in, ready to go. >> Fantastic. Is it powering on? And do you see anything on the screen? >> Yeah. So you can see here it is still

■ [25:06 - 25:26]



```
agent.py X
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
33         llm=llm.FallbackAdapter(
34             [
35                 inference.LLM(model="openai/invalid-model"),
36                 inference.LLM(model="google/gemini-2.5-flash"),
37             ]
38         ),
39         tts=tts.FallbackAdapter(
Problems Output Debug Console Terminal Ports Postman Console
181, in _traceable_main_task
    await self._main_task()
File
"/Users/jesse/workspace/agents-101-workshop/workshop-demo/livekit-voice-agent/.venv/lib/python3.11/site-pa
200, in _main_task
    return await self._run()
~~~~~
File
"/Users/jesse/workspace/agents-101-workshop/workshop-demo/livekit-voice-agent/.venv/lib/python3.11/site-pa
line 344, in _run
    raise APIStatusError(
livekit.agents._exceptions.APIStatusError: error creating provider: model not found: openai/invalid-model
INFO:livekit.agents:shutting down worker
19:56:04.658 INFO livekit.agents shutting down worker {"id": "unregistered"
cursor: agent to run Agent - KK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 35, Col 61 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

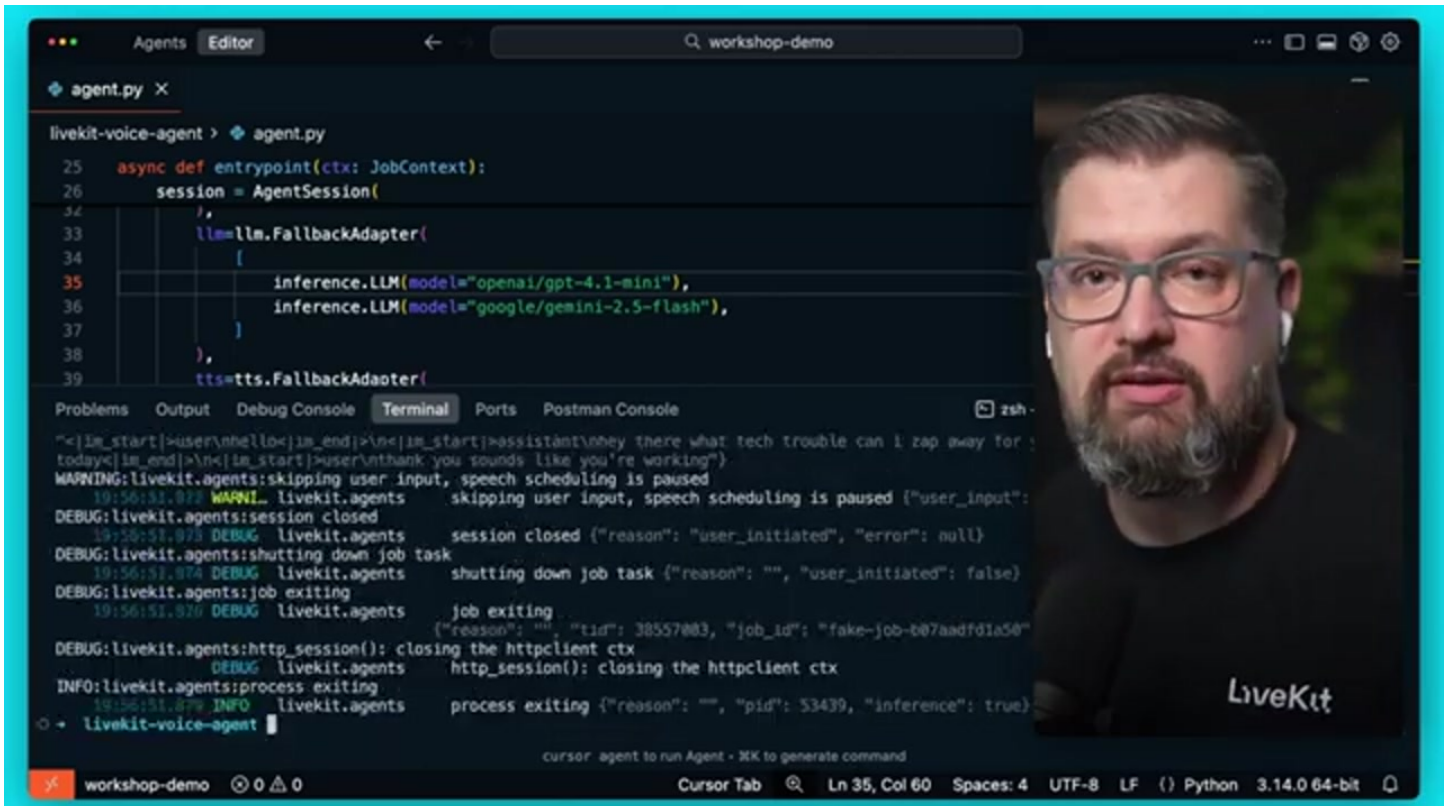
>> Yeah. So you can see here it is still working. Uh but we are of course getting errors which is good. We we love to see why it's not working. Uh but it is falling back to Gemini 2.5 because it obviously sees that OpenAI/invalid model is an invalid model. So of course uh we change this back to GPT41 mini.

■ [25:26 - 25:46]



uh we change this back to GPT41 mini. Save that. And let's go ahead and restart this. and it worked just fine with no errors. Hello. >> Hey there. What tech trouble can I zap away for you today? >> Oh, thank you. Sounds like you're working. So, this kind of chaos testing

■ [25:46 - 26:06]



The screenshot shows a VS Code editor window titled "Agents Editor" with a search bar containing "workshop-demo". The file "agent.py" is open, showing a Python script for a LiveKit agent. The script defines an entrypoint function that initializes an AgentSession with a fallback adapter for LLM and TTS. The terminal output shows the agent running and handling a user input, with various debug and warning messages.

```
agent.py X
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
34         llm=llm.FallbackAdapter(
35             inference.LLM(model="openai/gpt-4.1-mini"),
36             inference.LLM(model="google/gemini-2.5-flash"),
37         ),
38     ),
39     tts=tts.FallbackAdapter(
Problems Output Debug Console Terminal Ports Postman Console
"(<[is_start]>user\hello<[is_end]>\n<[is_start]>assistant\hey there what tech trouble can i zap away for :
today<[is_end]>\n<[is_start]>user\thank you sounds like you're working")
WARNING:livekit.agents:skipping user input, speech scheduling is paused
19:56:51.979 WARNING livekit.agents skipping user input, speech scheduling is paused {"user_input":
DEBUG:livekit.agents:session closed
19:56:51.979 DEBUG livekit.agents session closed {"reason": "user_initiated", "error": null}
DEBUG:livekit.agents:shutting down job task
19:56:51.979 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}
DEBUG:livekit.agents:job exiting
19:56:51.979 DEBUG livekit.agents job exiting {"reason": "", "tid": 38557083, "job_id": "fake-job-b07aadfd1a50"}
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
19:56:51.979 DEBUG livekit.agents http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
19:56:51.979 INFO livekit.agents process exiting {"reason": "", "pid": 53439, "inference": true}
livekit-voice-agent
```

working. So, this kind of chaos testing helps you confirm the agent will stay online even when a provider goes down. So, your agent now has a defined personality through the system prompt, a custom voice through the TTS configuration, and fallback adapters to handle provider outages. In the next lesson, you'll capture metrics so that you can measure performance and enable

- ✓ System persona
- ✓ Custom voice
- ✓ Fallback adapters



LIVEKIT.IO

you can measure performance and enable preemptive generation to reduce response latency. So, your agent now sounds great